

In this journal entry, I will talk about my thoughts and experiences with module 9. This lab taught me about 3 different things: the wisdom of the crowd, which, instead of passing the dataset through one Machine Learning model, passes it through multiple models to combine them and get a better result. It also includes boosting and the deep decision tree. These different tools showed me new ways to train a machine learning model based on the situation you have with your dataset.

The first thing I was shown in this lab was the Wisdom of the Crowd concept. As mentioned before, this method uses multiple machine learning models instead of just one. The thought behind this is that if different machine learning models make different mistakes, then each different model learns something, and what it gets wrong can be filled by another model. In this method, there can be multiple ways to do this; for example, bagging, where multiple machine learning models work at the same time. This lets them combine input, canceling out any random information and fixing an overfitting problem; this is where you would want to use a random forest. I like to think about it as a system of equations. If you have one person who is great at substitution, but they never use elimination, this can make it much more difficult if the system is asking for an elimination problem, as you can technically do it using substitution, but you are overfocusing too much one thing, which in a machine learning model would lead to overfitting. Instead, if you had 3 people who knew how to do both substitution and elimination but not as well as the one person, the group would be a better machine learning model, as when you combine their talents, they can solve either problem in their intended way and much more quickly.

The next way to do this is by using boosting. This is when machine learning models are placed in sequential order to fix any mistakes made by the previous model. Think about hiring 3 different managers for a failing restaurant to reduce complaints. The first manager comes in and looks at the current complaints and notices that a majority of people are getting undercooked food. He realizes this is due to bad kitchen equipment, so he can increase their budget, and now they can perfectly cook the food. The next manager comes in, and now that cooking times have increased, he notices complaints about slow service. He fixes this by having an alert system for the server when the food for a certain table is ready in their section. Now the restaurant has mostly dealt with all the major complaints, and Manager 3 is just there to help fix the restaurant's

smaller complaints, maybe about wait times or portion size. Notice how each manager only deals with the complaints still left behind from the last manager; this is boosting where the next machine learning model only fixes the issue of the last.

Finally, I would like to talk about the single deep decision tree. This is the opposite of the 2 examples I just gave you; now, instead of having it split up into multiple broad questions, it has very specific questions to determine the outcome. An example that came to mind was cooking a meal with exact recipes, but the way to measure them is only possible in that kitchen. Think if you go to a kitchen and they teach you how to cook using their equipment, and after you're done, you go back home to make the recipe at home, but the measurements are in their cups. For example, it says take the 3rd upper cabinet cup and fill it halfway with sugar. Technically this is right, but because it's so specific to the environment, it can't be done using anything else. This is exactly like a deep decision tree, as sometimes it becomes so specific that it can only work on a specific dataset and becomes useless anywhere else.